

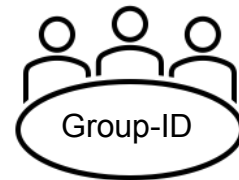
Suggested Project Topics for Distributed Systems Course (2026)

1. Telco-Edge CDN for Latency-Sensitive Video Streaming

Develop a content delivery network (CDN) optimized for telecom operators. The system caches video content at edge nodes to reduce latency and offload traffic from the core network.

Key Increments (6-Week Outline):

- 1. Introduction:** Basic client-server for video streaming.
- 2. Communication:** Implement edge–core communication for content requests and updates.
- 3. Time & Consensus:** Maintain consistent cache indices across edge nodes via consensus.
- 4. Performance & Scalability:** Introduce load balancing between edge nodes to handle high streaming demands.
- 5. Replication & Consistency:** Replicate streaming segments with eventual consistency for quick cache updates.
- 6. Resource Scheduling:** Allocate edge cache capacity based on content popularity.
- 7. System Architecture:** Separate streaming controller (microservice), caching logic (microservice), and client request handler.
- 8. Middleware:** Logging and analytics middleware to track popular content, QoS metrics, and user sessions.
- 9. Datacenter & Cloud:** Deploy in a telco datacenter environment with simulated network topologies.
- 10. Big Data Processing:** Process logs in a batch job to extract content popularity patterns.
- 11. Distributed ML:** Predict video demand spikes in near-real-time using a distributed ML model/LLM that analyzes aggregated logs across edge nodes.
- 12. Edge–Cloud Continuum:** Seamlessly offload large-scale content pre-caching to the cloud while local streams stay at the edge.



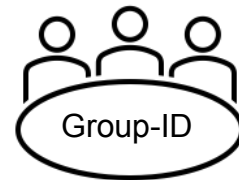
2. 5G-based IoT Gateway with Edge ML

Build a distributed **5G IoT gateway** that aggregates and processes sensor data from thousands of IoT devices.

Incorporate **Edge ML/LLM** to make real-time predictions (e.g., anomaly detection) close to the source.

Key Increments (6-Week Outline):

- 1. Introduction:** Simple IoT gateway collecting sensor data, storing it in a single node.
- 2. Communication:** Add MQTT or CoAP for device-to-gateway communication; REST/gRPC for gateway-to-cloud.
- 3. Time & Consensus:** Coordinate edge nodes with consistent configuration (e.g., over-the-air updates).
- 4. Performance & Scalability:** Load balancing for incoming IoT data streams and auto-scale gateways.
- 5. Replication & Consistency:** Edge gateway cluster replicates sensor data with eventual consistency.
- 6. Resource Scheduling:** Distribute ML inference tasks among edge gateways and central cloud.
- 7. System Architecture:** Microservices for device management, data ingestion, ML inference, and coordination.
- 8. Middleware:** Manage authentication, device provisioning, and logging in one layer.
- 9. Datacenter & Cloud:** Run batch analytics and heavier ML training in a telco cloud environment.
- 10. Big Data Processing:** Implement MapReduce/Spark to handle large historical sensor datasets.
- 11. Distributed ML:** Real-time anomaly detection model at the edge, updated from cloud training.
- 12. Edge-Cloud Continuum:** Adaptive scheduling so immediate analytics run at the edge, while deeper insights come from the cloud.

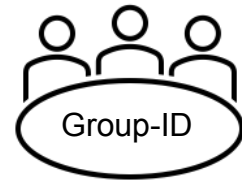


3. Telecom Fraud Detection & Prevention System

Develop a distributed solution for telecom **fraud detection** (e.g., SIM-box fraud, call spoofing). The system processes call detail records (CDRs) and network events in real time to flag suspicious activity.

Key Increments (6-Week Outline):

- 1. Introduction:** Basic ingestion pipeline for CDRs from various sources.
- 2. Communication:** Implement a robust event bus (Kafka or RabbitMQ) to stream CDRs to multiple analysis nodes.
- 3. Time & Consensus:** Ensure correct event ordering in multi-node scenarios for real-time detection.
- 4. Performance & Scalability:** Automatically scale analysis nodes when data spikes.
- 5. Replication & Consistency:** Maintain replicated state across detection engines with a consistency model.
- 6. Resource Scheduling:** Dynamically allocate detection jobs to nodes based on CPU load and memory usage.
- 7. System Architecture:** Decompose into microservices for ingestion, stream processing, and rule-based detection.
- 8. Middleware:** Centralize user authentication, anomaly alerts, and logging.
- 9. Datacenter & Cloud:** Deploy in a telco environment with both on-prem and cloud resources.
- 10. Big Data Processing:** Use batch processing to refine fraud detection rules on large historical CDR datasets.
- 11. Distributed ML:** Incorporate distributed ML models, e.g., random forest or deep learning, or LLM to detect fraud patterns.
- 12. Edge-Cloud Continuum:** Run light detection rules at edge nodes for immediate blocking, while advanced analytics happen in the cloud.

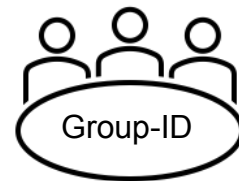


4. Edge-based RAN Analytics with Cloud Offloading

Create an **edge-based analytics** system for radio access networks (RAN). Collect and process RAN metrics (signal strength, user throughput) at the edge and offload complex jobs to the cloud for deeper insights.

Key Increments (6-Week Outline):

1. **Introduction:** Basic RAN data collection from simulated eNodeB/gNodeB.
2. **Communication:** Implement efficient protocols for node-to-edge data transfer (e.g., gRPC, MQTT).
3. **Time & Consensus:** Ensure consistent RAN performance data across multiple edge nodes.
4. **Performance & Scalability:** Add more edge analytics nodes to handle large geographic coverage.
5. **Replication & Consistency:** Mirror data between edge nodes in case of node failures.
6. **Resource Scheduling:** Prioritize real-time RAN metrics tasks vs. batch analytics.
7. **System Architecture:** Split into microservices (data collector, analytics engine, aggregator).
8. **Middleware:** Provide unified security, logging, and fallback mechanisms for edge devices.
9. **Datacenter & Cloud:** Offload heavier analytics (e.g., coverage heatmaps) to the cloud.
10. **Big Data Processing:** Use MapReduce or Spark for large-scale RAN performance analysis.
11. **Distributed ML:** Predict coverage holes or traffic hotspots with collaborative ML across edge nodes.
12. **Edge–Cloud Continuum:** Real-time RAN analytics happen at the edge; sophisticated training tasks in the cloud.

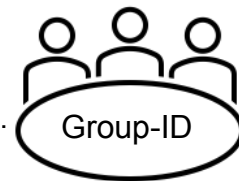


5. Geo-Distributed Telecom Data Processing (CDRs, Logs, Analytics)

Develop a geo-distributed processing system for **call detail records (CDRs)**, **logs**, and **operational analytics** that originate from diverse geographic locations. Focus on data consistency and low-latency access.

Key Increments (6-Week Outline):

- 1. Introduction:** Single-region aggregator for CDRs.
- 2. Communication:** Introduce an event bus or queue (Kafka) for cross-region data transfer.
- 3. Time & Consensus:** Keep a global ordering of events with logical or vector clocks.
- 4. Performance & Scalability:** Add more regional aggregators to handle higher data volumes.
- 5. Replication & Consistency:** Implement multi-region replication with conflict resolution or strong consistency per region.
- 6. Resource Scheduling:** Automatically schedule processing tasks in the region closest to the data source.
- 7. System Architecture:** Microservices for data ingestion, transformation, analytics, and coordination.
- 8. Middleware:** Provide centralized access control, encryption, and logging for multi-region data.
- 9. Datacenter & Cloud:** Simulate or use multiple data centers to store data locally before global aggregation.
- 10. Big Data Processing:** Perform large-scale batch analytics (e.g., usage trends, KPI reports) across all regions.
- 11. Distributed ML:** Collaboratively train global ML models using region-specific datasets.
- 12. Edge-Cloud Continuum:** Deploy partial ingestion at edge nodes for local data filtering, then send aggregated results to the cloud.

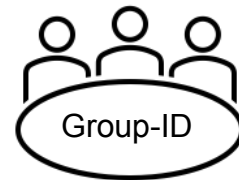


6. Data-Driven Resource Scheduling for 5G/6G Networks

Build a **scheduling engine** that optimally allocates spectrum, compute, and storage across multiple base stations or nodes based on data-driven insights. This includes dynamic TDD (Time Division Duplex) or bandwidth partitioning.

Key Increments (6-Week Outline):

1. **Introduction:** Single-node scheduler for assigning resources to users.
2. **Communication:** gRPC-based interface between scheduler and base stations.
3. **Time & Consensus:** Ensure consistent scheduling decisions across multiple base stations with a consensus protocol.
4. **Performance & Scalability:** Introduce additional scheduling nodes that horizontally scale to manage large networks.
5. **Replication & Consistency:** Keep the scheduling state (network usage, user QoS) replicated.
6. **Resource Scheduling:** Incorporate real-time metrics (signal quality, user load) for dynamic allocation.
7. **System Architecture:** Break down into microservices (data collector, scheduler, orchestration).
8. **Middleware:** Centralize logging of resource usage, authentication for network operators, and policy enforcement.
9. **Datacenter & Cloud:** Deploy scheduling service in the cloud, communicating with on-site RAN via secure tunnels.
10. **Big Data Processing:** Analyze historical scheduling data to spot patterns in user demand and throughput.
11. **Distributed ML:** Train ML models (e.g., reinforcement learning) to optimize scheduling decisions.
12. **Edge-Cloud Continuum:** Place smaller schedulers at edge sites for immediate decisions; coordinate with a global cloud-based scheduler.

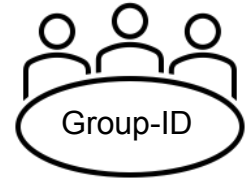


7. Telecom Digital Twin for Network Simulation & Monitoring

Design a **digital twin** of a telecom network to simulate and monitor real-time network conditions. The system mirrors the physical network in software, aiding in planning, fault analysis, and optimization.

Key Increments (6-Week Outline):

1. **Introduction:** Basic single-node simulator that models a few network parameters (bandwidth, latency).
2. **Communication:** Deploy a message bus for real-time telemetry (network stats) from live nodes to the digital twin.
3. **Time & Consensus:** Align real network events with simulated events for consistency.
4. **Performance & Scalability:** Scale the twin to handle large, complex topologies (multiple base stations, core network).
5. **Replication & Consistency:** Maintain mirrored states (live vs. twin) in a distributed manner.
6. **Resource Scheduling:** Simulate resource scheduling strategies within the digital twin and compare with real data.
7. **System Architecture:** Break into microservices for data collection, simulation engine, and analytics.
8. **Middleware:** Provide centralized logging, authentication, and data transformation modules.
9. **Datacenter & Cloud:** Run the digital twin platform in a telco cloud environment for high compute power.
10. **Big Data Processing:** Analyze historical twin vs. real-world performance data to refine simulation accuracy.
11. **Distributed ML:** Predict future network states or failures using a distributed ML pipeline on twin data.
12. **Edge–Cloud Continuum:** Incorporate real-time data from edge nodes to enhance the twin's accuracy and handle local network simulations.

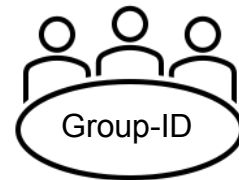


8. Pandemic Response & Contact Tracing System

Pandemic response requires fast, large-scale data gathering and analysis. Build a distributed platform for real-time contact tracing and outbreak modeling. Novelty: integrates multiple data sources (mobile apps, hospital records) to quickly identify infection chains.

Key Increments (6-Week Outline):

- 1. Introduction:** Single-node contact tracing simulation with minimal data.
- 2. Communication:** Build secure endpoints for app check-ins, hospital reports, lab results.
- 3. Time & Consensus:** Keep infection timeline consistent across the system.
- 4. Non-Functional Requirements:** High ingestion rate, privacy, data encryption.
- 5. Replication & Consistency:** Replicate logs for robust data availability.
- 6. Resource Management & Scheduling:** Distribute contact tracing tasks among cluster nodes.
- 7. System Architecture:** Microservices for data collection, analytics, notifications.
- 8. Middleware:** Security compliance (HIPAA-like), auditing logs.
- 9. Datacenter & Cloud:** Scale infrastructure to handle surges in infection reports.
- 10. Big Data Processing:** Offline analytics to detect transmission hotspots.
- 11. Distributed ML:** Predict outbreak trends or high-risk clusters.
- 12. Edge-Cloud Continuum:** Edge integration with local health clinics, cloud for national-level analytics.

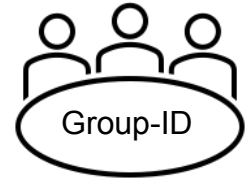


9. Distributed Electronic Health Record (EHR) System

Healthcare data often resides in siloed systems, making it difficult to unify patient records. Build a secure, distributed EHR system allowing multiple hospitals to share patient data while preserving privacy. Key technical novelties include: consensus and replication strategies for ensuring data integrity across facilities. Main challenge would in ensuring interoperability and reliability in a distributed environment for EHR setting.

Key Increments (6-Week Outline):

- 1. Introduction:** Single hospital EHR with basic Create, Read, Update, and Delete (CRUD) for patient data.
- 2. Communication:** Enable secure inter-hospital data exchange.
- 3. Time & Consensus:** Keep records consistent with distributed consensus (Raft/Paxos).
- 4. Non-Functional Reqs:** Ensure system can handle large spikes in data queries while maintaining low latency.
- 5. Replication & Consistency:** Redundant storage across hospitals.
- 6. Resource Management & Scheduling:** Efficient data synchronization tasks.
- 7. Architecture:** Microservices for patient records, audit logs, identity & access management.
- 8. Middleware:** Data encryption, compliance checks, logging.
- 9. Datacenter & Cloud:** Possibly private cloud for healthcare data.
- 10. Big Data Processing:** Analyze aggregated EHR data for population health insights.
- 11. Distributed ML:** Predict patient risks or readmissions across hospitals in real time.
- 12. Edge-Cloud Continuum:** Local clinics act as edge nodes, synchronizing with central cloud stores.



10. Distributed Ransomware Detection & File Recovery

Ransomware attacks can cripple organizations. Build a system that continuously monitors file changes across a distributed environment, detecting ransomware encryption patterns in real time and rolling back changes from replicated backups. Novelty is real-time detection plus distributed snapshotting for quick recovery. Distributed replication, real-time anomaly detection, and robust recovery protocols would be major technical contribution

Key Increments (6-Week Outline):

1. **Introduction:** Simple single-node file monitor to detect suspicious file modifications.
2. **Communication:** Event streaming from multiple endpoints monitoring file changes.
3. **Time & Consensus:** Sequence file changes, consensus on system “clean” state snapshots.
4. **Non-Functional Reqs:** Low-latency detection, minimal overhead on file writes.
5. **Replication & Consistency:** Replicated file system or versioning store to restore from.
6. **Resource Management & Scheduling:** Dynamically allocate detection tasks as data volume grows.
7. **System Architecture:** Microservices: monitoring agents, detection engine, recovery manager.
8. **Middleware:** Secure logging, authentication, role-based restore privileges.
9. **Datacenter & Cloud:** Backup storage distributed across multiple data centers.
10. **Big Data Processing:** Offline training on normal file operation patterns.
11. **Distributed ML:** Real-time anomaly detection for malicious encryption signatures.
12. **Edge–Cloud Continuum:** Local edge monitoring for immediate threat detection, cloud-based global rollback orchestration.

